

Reviewer Pack — Minimal Local Induction Dimension of Quasi-Polynomial Growth...

Entry 71a5dbf90350 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 06:10:23 UTC

Minimal Local Induction Dimension of Quasi-Polynomial Growth and Communication Complexity Rank

Entry ID: 71a5dbf90350 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 06:10:23 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Local Induction Dimension) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For any given Boolean function f , the minimal local induction dimension (MILD) of its associated binary tree is linearly correlated with its communication complexity rank (CCR), such that $\text{MILD}(f) = \Theta(\text{CCR}(f))$.

Rationale (proposer's reasoning):

Local Induction Dimension measures the complexity of a space by quantifying how many points are needed to uniquely determine all other points. This concept can potentially provide insights into the complexity of computation tasks like communication complexity. A connection between MILD and CCR could reveal new structures in communication complexity problems.

Taxonomy category: `local_induction_dimension` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 06c06dab112ec25f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

To support the conjecture, the correlation coefficient between MILD and CCR must be ≥ 0.8 for all seeds, with a mean metric value ≤ 3 across seeds. To falsify, any seed results in a correlation coefficient < 0.5 or a single seed shows $\text{MILD} > 10$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "local induction dimension" AND "communication complexity rank" - "MILD" AND "matrix rank" - "Boolean function" AND "algebraic topology"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def binary_tree_from_function(f):
        n = int(math.log2(len(f)))
        tree = [[None] * (2 ** i) for i in range(n + 1)]
        tree[0][0] = f
        for level in range(n):
            for j in range(2 ** level):
                if tree[level][j] is not None:
                    left_child = tree[level][j] & 1
                    right_child = (tree[level][j] >> 1) & 1
                    tree[level + 1][2 * j] = left_child
                    tree[level + 1][2 * j + 1] = right_child
        return tree

    def local_induction_dimension(tree):
        n = len(tree)
        if n == 0:
            return 0
        points = set()
        for level in range(n):
            for i in range(2 ** level):
                if tree[level][i] is not None:
                    points.add((level, i))
        max_distance = 0
        for p1 in points:
            for p2 in points:
```

```

        distance = abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])
        if distance > max_distance:
            max_distance = distance
    return max_distance

def communication_complexity_rank(tree):
    n = len(tree)
    if n == 0:
        return 0
    rank = 0
    for level in range(n):
        for i in range(2 ** level):
            if tree[level][i] is not None:
                rank += 1
    return rank

def correlation(x, y):
    mean_x = sum(x) / len(x)
    mean_y = sum(y) / len(y)
    cov = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(len(x))) / len(x)
    std_x = math.sqrt(sum((x[i] - mean_x) ** 2 for i in range(len(x))) / len(x))
    std_y = math.sqrt(sum((y[i] - mean_y) ** 2 for i in range(len(y))) / len(y))
    return cov / (std_x * std_y)

n_values = [5, 10, 15, 20, 30, 40]
mild_values = []
ccr_values = []

for n in n_values:
    f = generate_boolean_function(n)
    tree = binary_tree_from_function(f)
    mild = local_induction_dimension(tree)
    ccr = communication_complexity_rank(tree)
    mild_values.append(mild)
    ccr_values.append(ccr)

correlation_coefficient = correlation(mild_values, ccr_values)
metric_value = abs(correlation_coefficient)
instances_tested = len(n_values)
n_max = max(n_values)
conjecture_holds = correlation_coefficient >= 0.5 and mild <= 10
counterexample = "" if conjecture_holds else "correlation_coefficient < 0.5 or MILD > 10"

return {
    "metric_name": "Correlation Coefficient",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

```

```

for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
    results.append(trial_result)

mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
elif any(not result["conjecture_holds"] for result in results) and min(result["metric_value"]
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.5 or MILD > 10\" f{first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c969c651.py", line 106, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c969c651.py", line 79, in run_trial
    tree = binary_tree_from_function(f)
           ~~~~~^~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c969c651.py", line 31, in binary_tree_from_function
    left_child = tree[level][j] & 1
                  ~~~~~^~~~~
TypeError: unsupported operand type(s) for &: 'list' and 'int'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the correlation coefficient between MILD and CCR for all seeds. | next: Review the code to identify the cause of the crash and fix it to ensure that the test can run to completion.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12459
2	preregistration	ollama_remote	glm4:latest	0	0	10514
3	novelty	ollama_remote	glm4:latest	0	0	24621
4	novelty	ollama_remote	glm4:latest	0	0	15670
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14501
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21319
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10807
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14508
9	judge	ollama_remote	glm4:latest	0	0	17652

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 142051 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/71a5dbf90350.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/71a5dbf90350.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/71a5dbf90350.tar.gz* (if generated)